

2. Quadratic Probing

$f(i) = i^2$; /* a quadratic function */

[Theorem] If quadratic probing is used, and the table size is **prime**, then a new element can always be inserted if the table is **at least half empty**.

Proof: Just prove that the first $\lceil \text{TableSize}/2 \rceil$ alternative locations are all **distinct**. That is, for any $0 \leq i \neq j \leq \lfloor \text{TableSize}/2 \rfloor$, we have

$$(h(x) + i^2) \% \text{TableSize} \neq (h(x) + j^2) \% \text{TableSize}$$

Suppose: $h(x) + i^2 = h(x) + j^2 \pmod{\text{TableSize}}$

then: $i^2 = j^2 \pmod{\text{TableSize}}$

$$(i + j)(i - j) = 0 \pmod{\text{TableSize}}$$

TableSize is prime $\Rightarrow$ either $(i + j)$ or $(i - j)$ is divisible by **TableSize**

Contradiction !

For any x , it has $\lceil \text{TableSize}/2 \rceil$ distinct locations into which it can go. If **at most** $\lfloor \text{TableSize}/2 \rfloor$ positions are taken, then an empty spot can always be found. ■

Note: If the table size is a prime of the form $4k + 3$, then the quadratic probing $f(i) = \pm i^2$ can probe the entire table.

Read Figures 7.15 - 7.16 for detailed representations and implementations of initialization.

```

Position Find ( ElementType Key, HashTable H )
{
    Position CurrentPos;
    int CollisionNum;
    CollisionNum = 0;
    CurrentPos = Hash( Key, H->TableSize );
    while( H->TheCells[ CurrentPos ] == Empty &&
           H->TheCells[ CurrentPos ].Element != Key ) {
        CurrentPos += 2 * ++CollisionNum - 1;
        if ( CurrentPos >= H->TableSize ) CurrentPos -= H->TableSize;
    }
    return CurrentPos;
}

```

What is returned?

```
void Insert ( ElementType Key, HashTable H )
{
    Position Pos;
    Pos = Find( Key, H );
    if ( H->TheCells[ Pos ].Info != Legitimate ) { /* OK to insert here */
        H->TheCells[ Pos ].Info = Legitimate;
        H->TheCells[ Pos ].Element = Key; /* Probably need strcpy */
    }
}
```

Question: How to delete a key?

Note: ① Insertion will be seriously slowed down if there are too many **deletions intermixed with insertions**.
② Although primary clustering is solved, *secondary clustering* occurs – that is, keys that hash to the same position will probe the same alternative cells.

3. Double Hashing

$f(i) = i * \text{hash}_2(x);$ /* $\text{hash}_2(x)$ is the 2nd hash function */

① $\text{hash}_2(x) \not\equiv 0$; ① make sure that all cells can be probed.

👉 **Tip:** $\text{hash}_2(x) = R - (x \% R)$ with R a prime smaller than TableSize, will work well.

Note: ① If double hashing is correctly implemented, simulations imply that the **expected** number of probes is almost the same as for a **random** collision resolution strategy.

② Quadratic probing does not require the use of a second hash function and is thus likely to be **simpler and faster** in practice.

Robin Hood Hashing - Pedro Celis (1986)



Robbing the rich in order to give to the poor.

Reduce variance in look-up cost of linear probing by robbing slots from *rich* keys and give to *poor* keys.

Minimize probe distance:

$$d(x) = (c(x) - h(x) + \text{TableSize}) \% \text{TableSize}$$

$$d(\text{rich}) < d(\text{poor})$$

current position

**acos atoi char define exp
ceil cos float**

- 👍 Same asymptotic growth as linear probing while having slightly lower expected probes.
- 👍 The expected maximum probe sequence length is bounded to $O(\log N)$.
- 👍 Predictable performance with high load factor tolerance.
- 👎 Slow insertion and complex deletion.
- 👎 Require storing probe distance.



bucket	x	$d(x)$
0	acos	0
1	atoi	1
2	char	0
3	ceil	1
4	cos	2
5	define	2
6	exp	2
7	float	2
8		
9		
10		
...		
25		

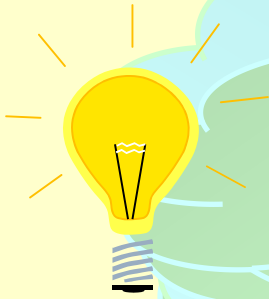
How to find **atan**?

```

bool RobinHood_Insert ( ElementType Key, HashTable H )
{
    Position Pos = Hash ( Key, H->TableSize );
    int CurrentDisp = 0;
    ElementType CurrentKey = Key;
    while ( true ) {
        if ( H->TheCells[Pos ].Info != Legitimate ) { /* Empty or Deleted */
            H->TheCells[ Pos ].Info = Legitimate;
            H->TheCells[ Pos ].Element = CurrentKey;
            H->TheCells[ Pos ].Disp = CurrentDisp;
            return true;
        }
        if ( CurrentDisp > H->TheCells[ Pos ].Disp ) { /* Rob the rich */
            swap CurrentKey and H->TheCells[ Pos ].Element
            swap CurrentDisp and H->TheCells[ Pos ].Disp
        }
        Pos = ( Pos + 1 ) % H->TableSize ;
        if ( ++CurrentDisp == H->TableSize ) return false; /* table is full */
    }
}

```

§ 5 Rehashing



- ➡ Build another table whose size is a prime and at least as twice as big;
- ➡ Scan down the entire original hash table for non-deleted elements; Then what can we do? ...
- ➡ Use a new function to hash those elements into the new table.

If there are N keys in the table, then $T(N) = O(N)$

Question: When to rehash?

Answer:

- ① As soon as the table is half full
- ② When an insertion fails
- ③ When the table reaches a certain load factor

Note: There should have been N insertions before rehash, so $O(N)$ rehash only adds a constant cost to each insertion.

However, in an interactive system, the unfortunate user whose insertion caused a rehash could see a slowdown.

**Read Figures 7.23
for detailed implementation of rehashing.**